Guide to Regularization, ML

@Fire2002

January 2026

1 Regularization Basics

Reference Link (notes from here taken below)

think of regularization as any method to prevent overfitting or help the optimization.

example of overfitting: polynomial repression

consider the function $t = \sin(2\pi x) + \epsilon$. $M = 9$ degrees of freedom.

E_{rms} vs. M would be (show graph):

Overfitting

- smaller dataset = larger difference between empirical and expected loss.

1.1 Regularization as a hard constraint

Training objective

$$\min_f \hat{L}(f) = \frac{1}{n}\sum_{i=1}^{n} l(f, x_i, y_i) \tag{1}$$

$$\text{subject to: } f \in \mathcal{H} \tag{2}$$

When parametrized, the problem becomes

$$\min_\theta \hat{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} l(\theta, x_i, y_i) \tag{3}$$

$$\text{subject to: } \theta \in \Omega \tag{4}$$

When Ω is measured by some quantity R

$$\min_\theta \hat{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} l(\theta, x_i, y_i) \tag{5}$$

$$\text{subject to: } ||\theta||_2^2 \leq r^2 \tag{6}$$

1.2 Regularization as a soft constraint

The hard-constraint optimization is equivalent to soft-constraint, s.t. for some regularization parameter $\lambda^* > 0$,

$$\min_\theta \hat{L}_R(\theta) = \frac{1}{n}\sum_{i=^1}^{n} l(\theta, x_i, y_i) + \lambda^* R(\theta) \tag{7}$$

Example: L_2 regularization

$$\min_\theta \hat{L}_R(\theta) = \frac{1}{n}\sum_{i=1}^{n} l(\theta, x_i, y_i) + \lambda^* ||\theta||_2^2 \tag{8}$$

Via the Lagrangian multiplier method,

$$\mathcal{L}(\theta, \lambda) \coloneqq \hat{L}(\theta) + \lambda[R(\theta) - r] \tag{9}$$

Suppose that θ^* is the optimal for hard-constraint optimization s.t.

$$\theta^* = \arg\min_\theta \max_{\lambda \geq 0} \mathcal{L}(\theta, \lambda) \coloneqq \hat{L}(\theta) + \lambda[R(\theta) - r] \tag{10}$$

And suppose that λ^* is the corresponding optimal for max s.t.

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta, \lambda^*) \coloneqq \hat{L}(\theta) + \lambda^*[R(\theta) - r] \tag{11}$$

1.3 Regularization as Bayesian prior

(from a Bayesian perspective, everything is a distribution)

- Prior over the hypotheses: $p(\theta)$
- Posterior over the hypotheses: $p(\theta|\{x_i, y_i\})$
- Likelihood: $p(\{x_i, y_i\}|\theta)$
- Bayesian rule:

$$p(\theta|\{x_i, y_i\}) = \frac{p(\theta)p(\{x_i, y_i\}|\theta)}{p(\{x_i, y_i\})} \tag{12}$$

- Maximum A Posteriori (MAP):

$$\max_\theta \log p(\theta|\{x_i, y_i\}) = \max_\theta \log p(\theta) + \log p(\{x_i, y_i\}|\theta) \tag{13}$$

  - (Regularization $\to \log p(\theta)$; MLE loss $\to \log p(\{x_i, y_i\}|\theta)$)
- Example: L_2 loss with L_2 regularization

$$\min_\theta \hat{L}_R(\theta) = \frac{1}{n}\sum_{i=1}^{n} (f_\theta(x_i) - y_i)^2 + \lambda^* ||\theta||_2^2 \tag{14}$$

  - Correspond to a normal likelihood $p(x, y|\theta)$ and a normal prior $p(\theta)$.

The typical choice for optimization is soft-constraint:

$$\min_\theta \hat{L}_R(\theta) = \hat{L}(\theta) + \lambda R(\theta) \tag{15}$$

Hard-constraint and the Bayesian view are typically for conceptual purposes or used for derivation.

Use cases:

- Hard-constraint if
  - given the explicit bound $R(\theta) \leq r$
  - soft-constraint causes trapped in a local minima with small θ
  - projection back to feasible set leads to stability
- Bayesian view if
  - given the prior distribution

1.4 Examples

1.4.1 L_2 regularization

$$\min_\theta \hat{L}_R(\theta) = \hat{L}(\theta) + \frac{\alpha}{2}||\theta||_2^2 \tag{16}$$

- Effect on (stochastic) gradient descent
- Effect on the optimal solution

Gradient of regularized objective

$$\nabla \hat{L}_R(\theta) = \nabla \hat{L}(\theta) + \alpha\theta \tag{17}$$

Gradient descent update

$$\theta \leftarrow \theta - \eta \nabla \hat{L}_R(\theta) = \theta - \eta \nabla \hat{L}(\theta) - \eta\alpha\theta = (1-\eta\alpha)\theta - \eta\nabla\hat{L}(\theta) \tag{18}$$

Terminology: weight decay

Effect on the optimal solution

Consider a quadratic approximation around θ^*

$$\hat{L}(\theta) \approx \hat{L}(\theta^*) + (\theta-\theta^*)^T \nabla \hat{L}(\theta^*) + \frac{1}{2}(\theta-\theta^*)^T H(\theta-\theta^*) \tag{19}$$

Since θ^* is optimal, $\nabla\hat{L}(\theta^*) = 0$

$$\hat{L}(\theta) \approx \hat{L}(\theta^*) + \frac{1}{2}(\theta-\theta^*)^T H(\theta-\theta^*) \tag{20}$$

$$\nabla\hat{L}(\theta) \approx H(\theta-\theta^*) \tag{21}$$

Gradient of regularized objective

$$\nabla\hat{L}_R(\theta) \approx H(\theta-\theta^*) + \alpha\theta \tag{22}$$

On the optimal θ_R^*

$$0 = \nabla\hat{L}_R(\theta_R^*) \approx H(\theta_R^* - \theta^*) + \alpha\theta_R^* \tag{23}$$

$$\theta_R^* = \approx (H+\alpha I)^{-1} H\theta^* \tag{24}$$

The optimal is

$$\theta_R^* = \approx (H+\alpha I)^{-1} H\theta^* \tag{25}$$

Suppose H has eigen-decomposition $H = Q\Lambda Q^T$

$$\theta_R^* \approx (H+\alpha I)^{-1} H\theta^* = Q(\Lambda+\alpha I)^{-1}\Lambda Q^T\theta^* \tag{26}$$

Effect: rescale along eigenvectors of H.

1.4.2 L_1 regularization

$$\min_\theta \hat{L}_R(\theta) = \hat{L}(\theta) + \alpha||\theta||_1 \tag{27}$$

- Effect on (stochastic) gradient descent
- Effect on the optimal solution

Effect on gradient descent

Gradient of regularized objective

$$\nabla \hat{L}_R(\theta) + \nabla \hat{L}(\theta) + \alpha \operatorname{sign}(\theta) \tag{28}$$

Where sign applies to each element in θ.

The gradient descent update is

$$\theta \leftarrow \theta - \eta \nabla \hat{L}_R(\theta) = \theta - \eta \nabla \hat{L}(\theta) - \eta \alpha \operatorname{sign}(\theta) \tag{29}$$

Effect on the optimal solution

For a quadratic approximation around θ^*

$$\hat{L}(\theta) \approx \hat{L}(\theta^*) + (\theta - \theta^*)^T \nabla \hat{L}(\theta^*) + \frac{1}{2}(\theta - \theta^*)^T H (\theta - \theta^*) \tag{30}$$

Since θ^* is optimal, $\nabla \hat{L}(\theta^*) = 0$ so

$$\hat{L}(\theta) \approx \hat{L}(\theta^*) + \frac{1}{2}(\theta - \theta^*)^T H (\theta - \theta^*) \tag{31}$$

Further assuming that H is diagonal and positive ($H_{ii} > 0, \forall i$)

- not true in general but assume for getting some intuition

The regularized objective, ignoring constants, is

$$\hat{L}_R(\theta) \approx \sum_i \frac{1}{2} H_{ii} (\theta_i - \theta_i^*)^2 + \alpha |\theta_i| \tag{32}$$

The optimal θ_R^* is

$$(\theta_R^*)_i \approx \begin{cases} \max\left\{\theta_i^* - \frac{\alpha}{H_{ii}}, 0\right\} & \text{if} \quad \theta_i^* \geq 0 \\ \min\left\{\theta_i^* + \frac{\alpha}{H_{ii}}, 0\right\} & \text{if} \quad \theta_i^* < 0 \end{cases} \tag{33}$$

Effect: induce sparsity

Assume that H is diagonal. The compact expression for the optimal θ_R^* is then

$$(\theta_R^*)_i \approx \operatorname{sign}(\theta_i^*) \max\{|\theta_i^*| - \frac{\alpha}{H_{ii}}, 0\} \tag{34}$$

1.4.3 Bayesian view

- L_1 regularization corresponds to the Laplacian prior

$$p(\theta) \propto \exp(\alpha \sum_i |\theta_i|) \tag{35}$$

$$\log p(\theta) = \alpha \sum_i |\theta_i| + \text{constant} = \alpha ||\theta||_1 + \text{constant} \tag{36}$$

2 Regularization Methods for Neural Networks

Reference Link (notes from here taken below)

The goal of regularization is to reduce overfitting. This is typically achieved by reducing model capacity and/or reduction of the variance of the predictions.

Regularization can be understood as the process of adding information/changing the objective function to prevent overfitting.

Common Regularization Techniques for DNNs include

- Early stopping
- L_1/L_2 regularization (norm penalties)
- Dropout

Improving generalization performance can appear in many forms:

Subcategories include: (much of this can be covered in separate material)

1. Dataset
    - Data acquisition
    - Data augmentation
    - Label smoothing
    - Leveraging data
    - Leveraging related data
2. Architecture
    - Varying weight initialization
    - Activation functions
    - Residual layers
    - Knowledge distillation
3. Normalization
    - Input standardization
    - BatchNorm and variants
    - Weight standardization
    - Gradient centralization
4. Training loop
    - Adaptive learning rates
    - Auxiliary losses
    - Gradient clipping
5. Regularization
    - L1/L2 regularization
    - Early stopping
    - Dropout

Often, the best way to reduce overfitting is collecting more data.

For a data augmentation example, click **here**.

Reducing network capacity & early stopping

Early Stopping

1. Split your dataset into 3 parts (always recommended)
    - use the test set only once at the end (for an unbiased estimate of generalization performance)
    - use the validation accuracy for tuning (always recommended)

2. Early stopping (not very common anymore)
   - reduce overfitting by observing the training/validation accuracy gap during training and then stop at the "right" point.

L1/L2 regularization L1 → LASSO regression (example shown later) L2 → Ridge regression (Tikhonov regularization) (example shown later)

2.1 L_2 Regularization for Linear Models (e.g., Logistic Regression)

$$\text{Cost}_{\mathbf{w},\mathbf{b}} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(y^{[i]}, \hat{y}^{[i]}) \tag{37}$$

$$\text{L2-Regularized-Cost}_{\mathbf{w},\mathbf{b}} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(y^{[i]}, \hat{y}^{[i]}) + \frac{\lambda}{n}\sum_{j} w_j^2 \tag{38}$$

where

$$\sum_{j} w_j^2 = ||\mathbf{w}||_2^2 \tag{39}$$

and λ is a hyperparameter.

In PyTorch,

```
    Automatically:
##########################################################
## Apply L2 regularization
optimizer = torch.optim.SGD(model.parameters(),
                            lr=0.1,
                            weight_decay=LAMBDA)
#-------------------------------------------------------
for epoch in range(num_epochs):
#### Compute outputs ####
    out = model(X_train_tensor)
#### Compute gradients ####
    cost = F.binary_cross_entropy(out, y_train_tensor)
    optimizer.zero_grad()
    cost.backward()
```

2.2 L_2 Regularization for Multilayer Neural Networks

$$\text{L2-Regularized-Cost}_{\mathbf{w},\mathbf{b}} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}(y^{[i]}, \hat{y}^{[i]}) + \frac{\lambda}{n}\sum_{l=1}^{L}||\mathbf{w}^{(l)}||_F^2 \tag{40}$$

where $||\mathbf{w}^{(l)}||_F^2$ is the Frobenius norm (squared):

$$||\mathbf{w}^{(l)}||_F^2 = \sum_{i}\sum_{j}(w_{i,j}^{(l)})^2 \tag{41}$$

2.3 L_2 Regularization for Neural Nets

The regular gradient descent update is

$$w_{i,j} \coloneqq w_{i,j} - \eta\frac{\partial\mathcal{L}}{\partial w_{i,j}} \tag{42}$$

The gradient descent update with L2 regularization is

$$w_{i,j} := w_{i,j} - \eta \left(\frac{\partial \mathcal{L}}{\partial w_{ij}} + \frac{2\lambda}{n} w_{i,j} \right) \tag{43}$$

In PyTorch,

```
# regularize loss
    L2 = 0.
    for name, p in model.named_parameters():
    if 'weight' in name:
            L2 = L2 + (p**2).sum()
    cost = cost + 2./targets.size(0) * LAMBDA * L2
    optimizer.zero_grad()
    cost.backward()
```

In short, the effects of norm penalties (regularization) can be severe:

- Large regularization $\implies$ high bias
- Low regularization $\implies$ high variance
- Assuming a nonlinear model $\implies$ good compromise

2.4 Dropout

2.4.1 Dropout in a Nutshell: Dropping Nodes

How do we drop the nodes practically/efficiently?

Bernoulli Sampling (during training): - $p :=$ drop probability - $v :=$ random sample from uniform distribution in range [0,1] - $\forall i \in v \,:\, v_i := 0$ if $v_i < p$ else 1 - $\mathbf{a} := \mathbf{a} \odot \mathbf{v}$.

After training when making predictions (during "inference"), scale activations via $\mathbf{a} := \mathbf{a} \odot (1 - p)$.

2.4.2 The Co-Adaptation Interpretation

Why does Dropout work well?

- Network will learn not to rely on particular connections too heavily
- Thus, will consider more connections (because it cannot rely on individual ones)
- The weight values will be more spread-out (may lead to smaller weights like with L2 norm)
- Side note: you can use different dropout probabilities in different layers (assigning them proportional to the number of units a layer is not a bad idea, for example)

2.4.3 The Ensemble Method Interpretation

- In dropout, we have a "different model" for each minibatch
- Via the minibatch iterations, we essentially sample over M=2h models, where h is the number of hidden units
- Restriction is that we have weight sharing over these models, which can be seen as a form of regularization
- During "inference" we can then average over all these models (but this is very expensive)

– This is basically just averaging log likelihoods (this is for one particular class):

$$p_{\text{Ensemble}} = \Big[\prod_{j=1}^{M} p^{\{i\}}\Big]^{1/M} = \exp\Big[1/M \sum_{j=1}^{M} log(p^{\{i\}})\Big] \tag{44}$$

(aka the "geometric mean")

For multiple classes, we need to normalize so that the probabilities sum to 1:

$$p_{\text{Ensemble},j} = \frac{p_{\text{Ensemble,j}}}{\sum_{j=1}^{k} p_{\text{Ensemble},j}} \tag{45}$$

2.4.4 Inverted Dropout

- Most frameworks implement inverted dropout
- Here, the activation values are scaled by the factor $(1 - p)$ during training instead of scaling the activators during "inference"
- I believe Google started this trend (because it's computationally cheaper in the long run if you use your model a lot after training)
- PyTorch's Dropout implementation is also inverted Dropout

In PyTorch,

```
class MultilayerPerceptron(torch.nn.Module):
    def __init__(self, num_features, num_classes, drop_proba,
            num_hidden_1, num_hidden_2):
        super().__init__()

        self.my_network = torch.nn.Sequential(
        # 1st hidden layer
        torch.nn.Flatten(),
        torch.nn.Linear(num_features, num_hidden_1),
        torch.nn.ReLU(),
        torch.nn.Dropout(drop_proba),
        # 2nd hidden layer
        torch.nn.Linear(num_hidden_1, num_hidden_2),
        torch.nn.ReLU(),
        torch.nn.Dropout(drop_proba),
        # output layer
        torch.nn.Linear(num_hidden_2, num_classes)
        )

    def forward(self, x):
        logits = self.my_network(x)
        return logits
```

Here, it is very important that you use 'model.train()' and 'model.eval()'!

```
for epoch in range(NUM_EPOCHS):
    model.train()
    for batch_idx, (features, targets) in enumerate(train_loader):
        features = features.view(-1, 28*28).to(DEVICE)
        ### FORWARD AND BACK PROP
        logits = model(features)
        cost = F.cross_entropy(logits, targets)
        optimizer.zero_grad()
```

```
            cost.backward()
            minibatch_cost.append(cost)
            ### UPDATE MODEL PARAMETERS
            optimizer.step()
        model.eval()
        with torch.no_grad():
            cost = compute_loss(model, train_loader)
            epoch_cost.append(cost)
            print('Epoch: %03d/%03d Train Cost: %.4f' % (
                epoch+1, NUM_EPOCHS, cost))
            print('Time elapsed: %.2f min' % ((time.time() - start_time)/60))
```

Tips for using dropout:

- Don't use Dropout if your model does not overfit
- However, you can increase the capacity to make the dataset overfit, then use dropout to be able to use a larger capacity model (but make it not overfit)

DropConnect: Randomly Dropping Weights

- Generalization of Dropout
- More "possibilities"
- Less popular and doesn't work so well in practice

For more on DropConnect, go see the following website **here**.

A secondary reading for Dropout can be found **here**.

3 Examples

reference link

3.1 LASSO Regression

In LASSO (Least Absolute Shrinkage and Selection Operator) regression, we add the absolute value of the magnitude of the coefficient as a penalty term to the loss function L.

The cost function is

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{i=1}^{m}|w_i| \tag{46}$$

Where

- m: number of features
- n: number of examples
- y_i: actual target value
- $\hat{y}_i$: theoretical target value

Implementation:

`X,y = make_regression(n_samples=100,n_features=5,noise=0.1,random_state=42)`

- generates a regression dataset with 100 samples, 5 features and some noise.

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

- splits the data into 80% training and 20% testing sets.

```
lasso = Lasso(alpha=0.1)
```

- creates a LASSO regression model with regularization strength alpha set to 0.1

3.2 Ridge Regression

In Ridge regression (L2 regularization technique), the cost function adds the squared magnitude of the coefficient as a penalty term to the loss function L. Ridge regression handles multicollinearity by shrinking coefficients of correlated features rather than eliminating them.

The cost function is

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{i=1}^{m} w_i^2 \tag{47}$$

Where

- n: number of examples/data points
- m: number of features
- y_i: actual target value for ith example
- $\hat{y}_i$: theoretical target value for ith example
- w_i: feature coefficients
- λ: regularization parameter to control the regularization strength

Implementation:

```
ridge = Ridge(alpha=1.0)
```

- creates a Ridge regression model with $\alpha = 1.0$.

3.2.1 Elastic Net Regression

In Elastic Net Regression (L1 + L2 regularization combination technique), we add the absolute norm of the weights as well as the squared measure of the weights.

The cost function here is

$$\text{Cost} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda((1-\alpha)\sum_{i=1}^{m}|w_i| + \alpha\sum_{i=1}^{m} w_i^2) \tag{48}$$

Where

- n: number of examples/data points
- m: number of features
- y_i: actual target value for ith example
- $\hat{y}_i$: theoretical target value for ith example
- w_i: feature coefficients
- λ: regularization strength parameter
- α: mixing parameter where $\alpha \in [0, 1]$ and $\alpha = 1$ for LASSO L_1 regularization, $\alpha = 2$ for Ridge L_2 regularization, and values between 0 and 1 yield a balance to L_1 and L_2 regularization.

Implementation

```
model = ElasticNet(alpha=1.0, l1_ratio=0.5)
```

- creates an Elastic Net model with $\alpha = 1.0$ and $L1/L2$ mixing ratio of 0.5.

For more information on regularization methods for model constraining, weight decay, and augmenting error, visit the link below: Regularization - Constraining the Model, Weight Decay, and Augmented Error